

# University of Lincoln

## Byte into Python: Getting started



Below are the step-by-step instructions for getting started with writing, running and testing Python code on the lab PCs. This knowledge will enable you to attempt the second two Byte into Python activities. If you are familiar with Python and speed through these instructions, please help those around you. You all have different background experiences and will all work through the activities at different rates.

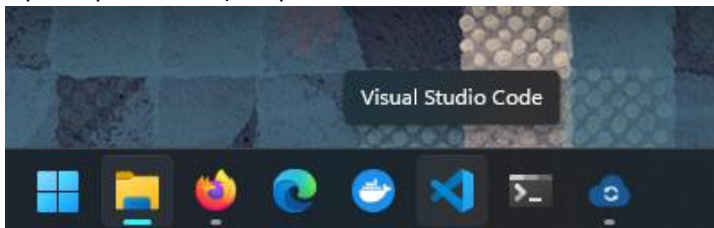
### Downloading and running the code

Getting the code:

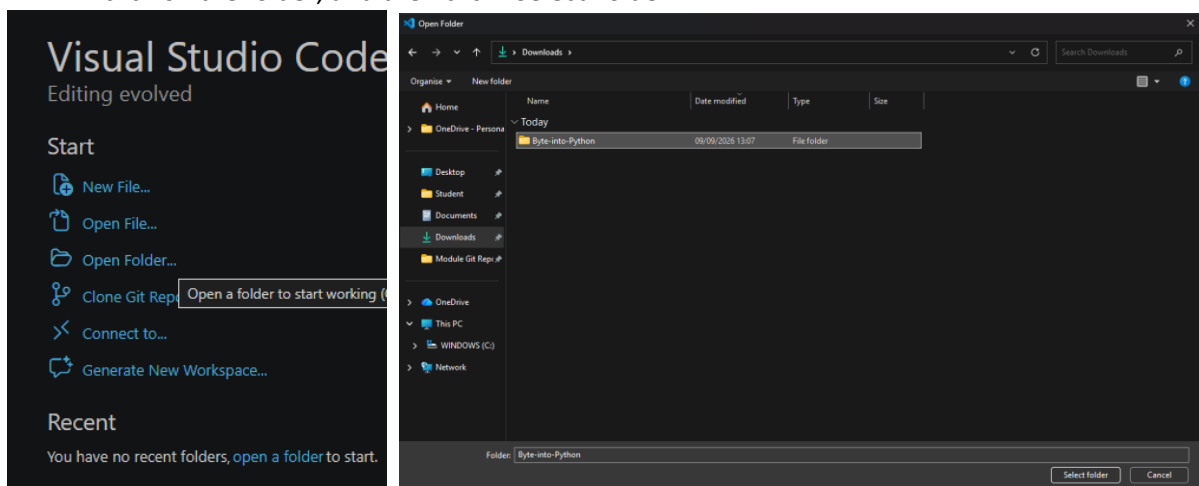
- Go to <https://github.com/HelenHarman/uol-ww-byteintopython>
- Click the green "<> Code" button, then "Download ZIP".
- Once downloaded unzip the folder.

Running the code via Visual Studio (VS) Code:

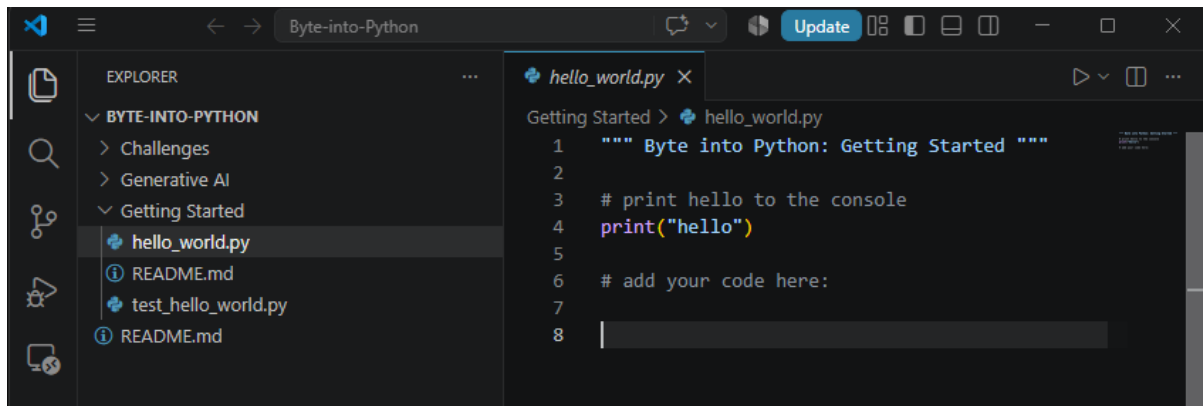
- Open-up VS Code (it is pinned to the windows bar at the bottom of your screen).



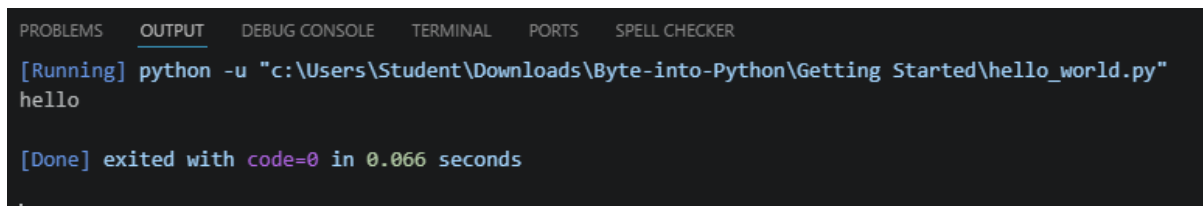
- Click open folder. Find the Byte into Python folder you downloaded (and unzipped). Single-click on the folder, and then click "Select Folder".



- Open the Getting Started/hello\_world.py file.



- Click the play symbol/button in the top right. You should see “hello” print to the OUTPUT window:



- Try modifying the print statement and re-running the code.
  - Note text between three quote marks and text after a # symbol are comments. Comments are used to document code and are ignored by the Python interpreter.

As well as writing to the console, we can read what the user types into the console.

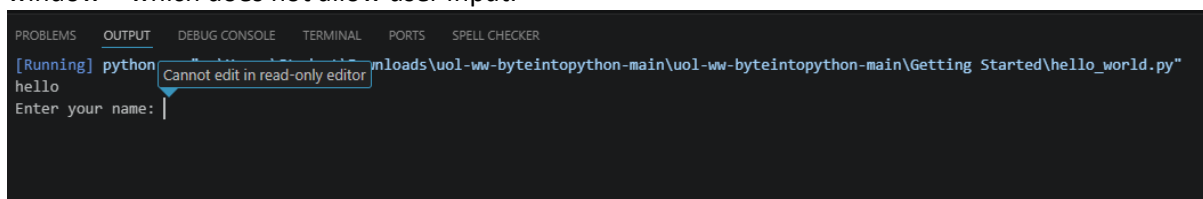
- Add the following code to the file and re-run the file.

```
name = "" # create a variable, containing an empty string

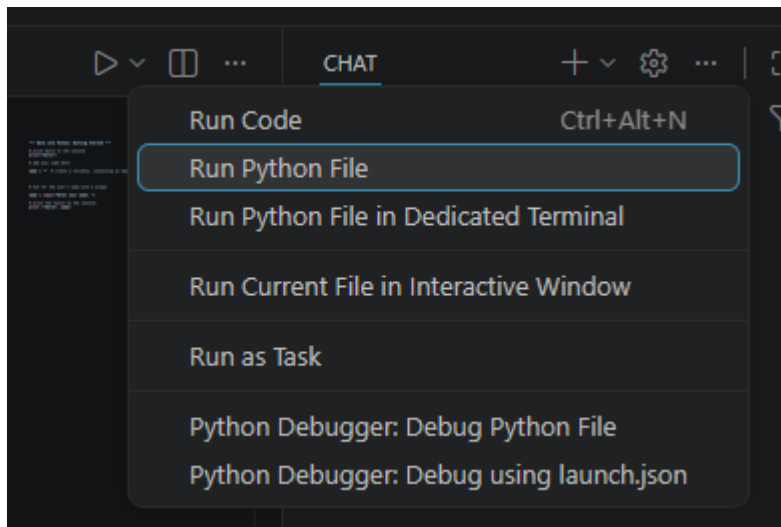
# Ask for the user's name with a prompt
name = input("Enter your name: ")

# print the result to the console:
print ("hello", name)
```

You should notice that you cannot enter a name. This is because the code is running in the OUTPUT window – which does not allow user input.



- Click the stop button in the top right.
- Next to the play button, is a small drop-down list arrow. Click this and then click “Run Python File.”



The code should now run in TERMINAL, which allows you to enter user input.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER

PS C:\Users\Student\Downloads\uol-ww-byteintopython-main> & C:\Users\Student\AppData\Roaming\uv\python\cpython\python.exe .\main.py
hello
Enter your name: Helen
hello Helen
PS C:\Users\Student\Downloads\uol-ww-byteintopython-main> |
```

## Adding your first function

A function is a reusable block of code that performs a specific task. Functions help avoid repetition, improve readability, and make code easier to maintain.

In Python, to define a function, use the **def** keyword followed by the function name, parentheses for parameters, and a colon. The function body must be indented. Below is an example of a function that takes two parameters and returns the sum of those two parameters.

```
def sum_values(a, b):
    return a + b
```

Once defined, we can call the function by using its name followed by parentheses.

```
first = 2 # create a variable and assign it the value 2
second = 3

# call the function and store the result in "result":
result = sum_values(first, second)

# print the result to the console:
print (result)

# The above lines could have been written in a single line;
# however, it can be clearer to split-up code into multiple lines
print(sum_values(2, 3))
```

- Copy the above two blocks of code into you file and run it.
- Trying to add a new call to the sum\_values function.

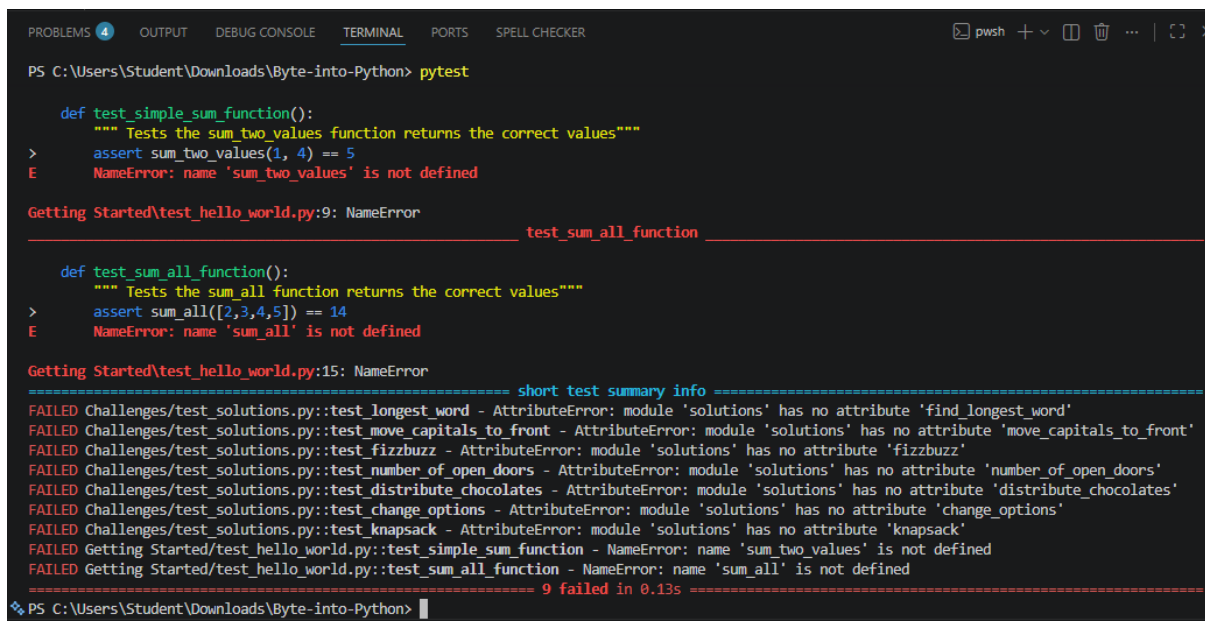
## Testing your function

Unit tests enable individual components or functions of a software application to be tested in isolation to ensure they perform as expected. They help developers to identify and fix bugs early in the development process, significantly improving code quality and reliability.

Before testing the functions, comment out or delete the `name = input("Enter your name: ")` line of code. Having user input (that is outside a function) will prevent the unit tests from running.

In Python, you can write unit tests using the pytest framework. We have developed a set of python unit tests for this activity and the "Challenges" activity. To run the python tests:

- Click on to the "TERMINAL" tab, type "pytest" and press enter.
- You should see that all the tests fail:



```
PS C:\Users\Student\Downloads\Byte-into-Python> pytest

def test_simple_sum_function():
    """ Tests the sum_two_values function returns the correct values"""
> assert sum_two_values(1, 4) == 5
E NameError: name 'sum_two_values' is not defined

Getting Started\test_hello_world.py:9: NameError

test_sum_all_function

def test_sum_all_function():
    """ Tests the sum_all function returns the correct values"""
> assert sum_all([2,3,4,5]) == 14
E NameError: name 'sum_all' is not defined

Getting Started\test_hello_world.py:15: NameError

===== short test summary info =====
FAILED Challenges/test_solutions.py::test_longest_word - AttributeError: module 'solutions' has no attribute 'find_longest_word'
FAILED Challenges/test_solutions.py::test_move_capitals_to_front - AttributeError: module 'solutions' has no attribute 'move_capitals_to_front'
FAILED Challenges/test_solutions.py::test_fizzbuzz - AttributeError: module 'solutions' has no attribute 'fizzbuzz'
FAILED Challenges/test_solutions.py::test_number_of_open_doors - AttributeError: module 'solutions' has no attribute 'number_of_open_doors'
FAILED Challenges/test_solutions.py::test_distribute_chocolates - AttributeError: module 'solutions' has no attribute 'distribute_chocolates'
FAILED Challenges/test_solutions.py::test_change_options - AttributeError: module 'solutions' has no attribute 'change_options'
FAILED Challenges/test_solutions.py::test_knapsack - AttributeError: module 'solutions' has no attribute 'knapsack'
FAILED Getting Started/test_hello_world.py::test_simple_sum_function - NameError: name 'sum_two_values' is not defined
FAILED Getting Started/test_hello_world.py::test_sum_all_function - NameError: name 'sum_all' is not defined
===== 9 failed in 0.13s =====
PS C:\Users\Student\Downloads\Byte-into-Python>
```

- Re-name the `sum_values` function to `sum_two_values`
  - Right-click on the function name, then click "Rename Symbol". This method of updating the name will update the function name everywhere you have used it.
- Add the following block of code to the `hello_world.py` file:

```
def sum_all(values):
    total = 0
    # loop through all values and add them to total
    for value in values:
        total = total + value
    return total

values = [3,4,5]
# You should call the sum_all function, pass it "values".
# Then print the result
```

- Try re-running pytest. You should now see that the two Getting Started tests pass.

```

===== short test summary info =====
FAILED Challenges/test_solutions.py::test_longest_word - AttributeError: module 'solutions' has no attribute 'find_longest_word'
FAILED Challenges/test_solutions.py::test_move_capitals_to_front - AttributeError: module 'solutions' has no attribute 'move_capitals_to_front'
FAILED Challenges/test_solutions.py::test_fizzbuzz - AttributeError: module 'solutions' has no attribute 'fizzbuzz'
FAILED Challenges/test_solutions.py::test_number_of_open_doors - AttributeError: module 'solutions' has no attribute 'number_of_open_doors'
FAILED Challenges/test_solutions.py::test_distribute_chocolates - AttributeError: module 'solutions' has no attribute 'distribute_chocolates'
FAILED Challenges/test_solutions.py::test_change_options - AttributeError: module 'solutions' has no attribute 'change_options'
FAILED Challenges/test_solutions.py::test_knapsack - AttributeError: module 'solutions' has no attribute 'knapsack'
===== 7 failed, 2 passed in 0.13s =====
PS C:\Users\Student\Downloads\Byte-into-Python>

```

During your "Challenges" activity, you will work towards getting the remaining tests to pass.

If you have finished the above steps, start looking at the Generative AI activity: <https://lncn.ac/WW-AI> When everyone has finished the above Getting Started steps, we will split you into groups to continue with the Generative AI activity and the Python Challenges.